# Kubernetes Lab Exam — Quick Recall Sheet

**Environment:** Windows CMD · Docker Desktop · Minikube · kubectl

## 1. Start and check the cluster

```
minikube start --driver=docker
kubectl config use-context minikube
kubectl get nodes
minikube status
```

If `kubectl` says it cannot connect to `localhost:8080`, check that Docker Desktop is running, start Minikube, and verify the current context.

## 2. Generate the three basic YAML files

Run in your lab working folder:

```
kubectl run mypod --image=nginx --dry-run=client -o yaml > pod.yaml
kubectl create deployment myapp --image=nginx --dry-run=client -o yaml >
deployment.yaml
kubectl create service nodeport myapp --tcp=80:80 --node-port=30080 --dry-
run=client -o yaml > service.yaml
```

- `pod.yaml`: one standalone Pod.
- `deployment.yaml`: a Deployment that manages Pods.
- `service.yaml`: exposes the app through a NodePort Service.

**Remember:** `--dry-run=client -o yaml` generates YAML without creating the resource. `>` saves the output to a file.

## 3. Inspect and edit files

```
type pod.yaml
type deployment.yaml
type service.yaml
notepad deployment.yaml
notepad service.yaml
```

Use spaces, not tabs, for YAML indentation. Keep indentation consistent.

## 4. Check labels and selectors

The Deployment generated above normally uses:

```
spec:
  selector:
    matchLabels:
      app: myapp
  template:
```

```
    metadata:
      labels:
        app: myapp
```

Make sure the Service selector matches the Deployment's Pod-template labels:

```
spec:
  selector:
    app: myapp
```

The standalone Pod uses `run: mypod`; it is separate from the Deployment and will not be selected by the `myapp` Service unless its labels and the Service selector are changed to match.

## 5. Deploy and verify

Start with the Deployment and Service:

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get deployments
kubectl get pods
kubectl get services
kubectl get all
```

To deploy the standalone Pod too:

```
kubectl apply -f pod.yaml
kubectl get pods
```

To validate a manifest before applying it:

```
kubectl apply --dry-run=client -f deployment.yaml
```

To inspect details:

```
kubectl describe deployment myapp
kubectl describe pod mypod
kubectl describe service myapp
```

## 6. Access the NodePort Service in Minikube

```
minikube service myapp --url
```

Open the URL it prints. If needed, use port forwarding instead:

```
kubectl port-forward service/myapp 8080:80
```

Then open `http://localhost:8080` in your browser while port-forwarding remains active.

## 7. Common operations

**Scale a Deployment:**

```
kubectl scale deployment myapp --replicas=3
kubectl get deployments
kubectl get pods
```

**View logs:**

```
kubectl get pods
kubectl logs POD_NAME
```

Replace `POD_NAME` with an actual Pod name from `kubectl get pods`.

**Delete resources created from files:**

```
kubectl delete -f service.yaml
kubectl delete -f deployment.yaml
kubectl delete -f pod.yaml
```

## 8. Quick command bank

| Task | Command |
|---|---|
| List Pods | `kubectl get pods` |
| List Deployments | `kubectl get deployments` |
| List Services | `kubectl get services` |
| List all common resources | `kubectl get all` |
| Detailed resource info | `kubectl describe RESOURCE NAME` |
| Apply one YAML file | `kubectl apply -f FILE.yaml` |
| Delete using a YAML file | `kubectl delete -f FILE.yaml` |
| Edit a file | `notepad FILE.yaml` |
| Show a file in CMD | `type FILE.yaml` |
| Scale Deployment | `kubectl scale deployment myapp --replicas=3` |

## 9. YAML concepts to recall

- **Pod** = runs container(s).
- **Deployment** = manages replicas and replaces failed Pods.
- **Service** = provides a stable way to reach matching Pods.
- **NodePort** = exposes a Service on a port of the node.
- `apiVersion` = Kubernetes API group/version.
- `kind` = resource type.
- `metadata` = name and labels.
- `spec` = desired configuration.
- `selector` = which Pods a resource targets.
- `template` = Pod definition used by a Deployment.
- `replicas` = desired number of Pods.
- `containerPort` documents the container's listening port; it does not by itself publish the app externally.

**Memory trick:** Pod = Run · Deployment = Manage · Service = Connect · Ingress = HTTP routing.

## 10. Common exam mistakes

1. Forgetting to start Docker Desktop/Minikube before deploying.
2. Using tabs or incorrect YAML indentation.
3. Mismatching Service selector labels and Deployment Pod-template labels.
4. Assuming `--dry-run=client` actually deploys resources—it only generates/validates output.
5. Using `kubectl expose deployment myapp` when `myapp` does not exist. That command normally requires an existing Deployment.
6. Expecting a standalone Pod and Deployment to be the same resource; they are separate.
7. Forgetting that `kubectl apply -f .` applies all applicable manifests in the current directory. Prefer a specific filename when unsure.

## Exam workflow

1. Start Minikube and check `kubectl get nodes`.
2. Generate the requested YAML file(s).
3. Inspect/edit the YAML and check labels/selectors.
4. Apply the manifest(s).
5. Verify with `kubectl get ...` and, if asked, `kubectl describe ...`.
6. Test access if a Service is part of the question.

**Core recall:** Generate → Inspect → Apply → Verify.